Federated Learning in SDVs: Optimizing Communication Frequency

Spyridon Arvanitis

I. Introduction

As connected devices become more widespread, artificial intelligence (AI) is increasingly being pushed closer to the network edge. In this setting, Federated Learning (FL) has gained attention as a practical approach for training models without directly sharing sensitive data. Instead of sending raw data to a central server, edge devices train models locally and only transmit the resulting parameters. This helps address privacy concerns while still making use of distributed computational resources. However, most traditional FL approaches assume stable and reliable communication, which is not always realistic—especially in highly dynamic environments.

At the same time, the automotive industry is moving toward Software-Defined Vehicles (SDVs). These vehicles effectively act as powerful, mobile computing nodes, running complex AI models to support functions such as autonomous driving, perception, and predictive maintenance. Since SDVs generate large volumes of sensor data on a daily basis, FL seems like a natural fit: it allows vehicles to collaboratively train shared models without constantly uploading raw, privacy-sensitive data to the cloud.

However, combining FL with SDV networks introduces a major challenge: communication overhead. Standard FL requires frequent exchanges of model parameters between clients and a central server during each training round. In a Vehicle-to-Everything (V2X) wireless environment, this quickly becomes problematic. Vehicles are constantly moving, connections are unstable, and available bandwidth is limited, especially when many vehicles try to communicate at the same time.

This leads to a highly complex, multi-dimensional tradeoff. Traditionally, FL research balances communication frequency against model consistency: frequent updates strain the V2X network, while sparse communication triggers 'model drift' due to the highly heterogeneous (non-IID) data collected by different vehicles. However, in an SDV, we must also account for internal compute constraints. SDVs utilize shared, virtualized compute platforms where FL must coexist safely with dynamic, high-priority workloads like Advanced Driver Assistance Systems (ADAS). Forcing a vehicle to perform intense local training to save bandwidth might steal crucial CPU cycles from safety-critical functions, creating an unacceptable risk.

Therefore, optimizing FL in SDVs is no longer just a network scheduling issue; it is a multi-objective, safety-constrained resource allocation problem. In this paper, we challenge the assumption that FL processes operate independently of vehicle architecture. Instead, we propose a deterministic, runtime policy engine that dynamically co-optimizes internal compute allocation (e.g., container CPU quotas), local training intensity, and external communication frequency. Our goal is to design a lightweight heuristic mechanism that strictly preserves functional safety and network efficiency, while simultaneously minimizing model divergence in dynamic V2X environments.

II. Related Work

The deployment of Federated Learning (FL) and edge computing in vehicular networks has attracted significant research interest due to its privacy-preserving nature. Existing literature can be broadly categorized into three main paradigms based on their resource modeling and optimization objectives: network-driven mobility management, joint compute-communication co-design, and compression/asynchronous transmission frameworks.

A. Network-Driven and Mobility-Aware Transmission Prior work has addressed wireless channel uncertainty and dynamic topologies inherent to vehicular environments. These approaches primarily treat vehicular nodes as communication links, optimizing transmission parameters while maintaining simplified or implicit models for onboard computation hardware. For instance, Hong et al. [1] formulated mobility-aware Vehicle-to-Infrastructure (V2I) model upload scheduling as a Markov Decision Process (MDP) to minimize upload delay and energy by optimizing transmission time and power. To counter imperfect Channel State Information (CSI) and vehicle mobility, Karatas et al. [2] proposed an adaptive rate and client selection scheme designed to accelerate FL convergence. Similarly, Yan et al. [3] introduced a dynamic scheduling framework that leverages opportunistic Vehicle-to-Vehicle (V2V) relays to maximize successful model uploads under strict energy constraints. While these strategies effectively mitigate wireless link degradation, their lack of explicit, granular hardware models limits their efficacy when vehicles possess highly heterogeneous or constrained onboard computation capabilities.

B. Joint Computation and Communication Co-Design To resolve the coupling between local processing latency and wireless transmission delay, joint compute-communication frameworks have been proposed. In this paradigm, onboard hardware variables—such as CPU frequency, local workloads, and batch sizes—are treated as active control variables. Qiang et al. [4] tackled the energy-delay tradeoff in Vehicular Edge Computing (VEC) by dynamically shifting

the model partition point based on the real-time availability of vehicle CPU resources and network bandwidth. Taking a predictive approach, Ballotta et al. [5] introduced a computation-scheduling co-design that utilizes mobility-aware radio environment maps to jointly optimize learning progress and local training workloads within a strict latency budget.

Resource allocation from a holistic perspective is also explored by Zhang et al. [6], who developed a joint framework for vehicle selection alongside compute-communication resource scheduling to maximize final model accuracy. Furthermore, Pervej et al. [7] provided a rigorous mathematical formulation coupling local iteration counts and CPU frequencies with wireless physical resource block allocation and sojourn time. While these joint formulations significantly improve resource efficiency, they typically assume a synchronous, uncompressed update framework, which can introduce severe bottlenecks when scaling to highly congested or deeply heterogeneous automotive networks.

C. Compressed and Asynchronous Learning Frameworks To alleviate bandwidth bottlenecks and mitigate the "straggler problem" caused by hardware or channel heterogeneity, researchers have investigated alternative communication protocols, such as quantization, compression, and asynchronous updating. Liu et al. [8] investigated the joint optimization of local batch sizes, spectrum allocation, and gradient compression ratios to maximize FL convergence rates under hard latency bounds. Addressing data packet sizing directly, Zhang et al. [9] proposed a heterogeneous quantized FL framework that dynamically tunes quantization levels and client selection based on mobility-aware transmission conditions.

To bypass the synchronization barriers of traditional FL, Wu et al. [10] developed an asynchronous update strategy where both training efforts and upload frequencies adapt fluidly to instantaneous link qualities, effectively suppressing redundant transmissions. Zhang et al. [11] demonstrated the real-world viability of asynchronous FL through an automotive case study utilizing heterogeneous embedded GPU platforms, proving that sliding-window data transfers could achieve rapid convergence with minimal bandwidth overhead.

D. Summary and Research Gap While the aforementioned literature successfully addresses various dimensions of vehicular learning, ranging from opportunistic networking to joint CPU bandwidth partitioning, a critical gap remains. Most existing works implicitly assume that a vehicle's onboard compute resources are either dedicated to or statically partitioned for the FL process. They fail to account for the mixed-criticality nature of modern Software-Defined Vehicles (SDVs), where FL is a secondary workload that must dynamically share a programmable compute platform with safety-critical automotive functions (e.g., perception, planning, and control). As outlined in Table 1, prior art does not simultaneously optimize network resources alongside dynamic, interference-aware compute allocation. To bridge this gap, this paper introduces a comprehensive problem formulation (detailed in Section III) that explicitly models the runtime compute occupancy of safety-critical tasks ($U_i^{\text{crit}}(t)$). By jointly optimizing compute allocation ($c_i(t)$), local training intensity ($e_i(t)$), and communication decisions ($p_i(t)$), our model uniquely navigates the trade-off between communication overhead, model divergence, and compute interference, ensuring that FL progresses efficiently without violating the strict compute constraints of the SDV platform.

III. Problem Formulation

We consider a federated learning (FL) system deployed over a set of Software-Defined Vehicles (SDVs), denoted by $\mathcal{V} = \{1, 2, \ldots, N\}$. Each vehicle operates as an FL client that locally trains a machine learning model using onboard data generated from sensors and driving operations. Periodically, vehicles transmit model updates to an edge/cloud server, which aggregates the received updates and distributes an updated global model.

Unlike conventional FL settings, SDVs are programmable compute platforms where multiple workloads coexist, including safety-critical automotive functions (e.g., perception, control, and planning). As a result, federated learning cannot be treated as an isolated task. Instead, FL computation must share system resources with other vehicle functions and adapt to runtime conditions such as compute availability and network connectivity.

To capture this interaction, we formulate a joint communication–compute scheduling problem that dynamically adjusts FL participation decisions based on system state and quantifiable learning gains. To enable online adaptation to dynamic network and system conditions, time is divided into discrete decision intervals, indexed by $t \in \{0, 1, \ldots, T-1\}$.

III.1. System State

At each time interval t, vehicle i observes a local system state defined as:

$$S_i(t) = \{B_i(t), U_i^{\text{crit}}(t), D_i(t)\} \tag{1}$$

where:

- $B_i(t)$ denotes the available uplink bandwidth. This parameter captures the quality of the wireless communication channel between the vehicle and the edge/cloud server.
- $U_i^{\text{crit}}(t)$ represents the fraction of compute resources currently occupied by safety-critical workloads. Since these tasks have strict real-time requirements, FL computation must operate only on the remaining available resources.
- $D_i(t)$ denotes the model divergence (drift indicator), which measures how far the local model has deviated from the latest global model.

These state variables jointly capture the network condition, runtime resource usage, and learning state of each vehicle.

III.2. System Costs and Learning Dynamics

The system state variables map directly to the operational costs and learning performance metrics of the FL process. We define these components as follows:

III.2.1. Communication Cost. Communication overhead increases when vehicles transmit model updates, mapping to the available bandwidth state $B_i(t)$:

$$C_{\text{net}}(t) = \sum_{i \in \mathcal{V}} p_i(t) \frac{M_i}{B_i(t)} \tag{2}$$

where M_i denotes the size of the transmitted model update.

III.2.2. Compute Interference Cost. FL computation may interfere with safety-critical workloads. This cost is a direct function of the occupied compute fraction $U_i^{\text{crit}}(t)$:

$$C_{\text{comp}}(t) = \sum_{i \in \mathcal{V}} c_i(t) U_i^{\text{crit}}(t) \tag{3}$$

III.2.3. Drift Dynamics and Cost. Model divergence affects learning accuracy and convergence, especially in non-IID data distributions. Instead of theoretical accumulation, drift $D_i(t)$ is explicitly measured as the Euclidean distance between the local weights and the global weights:

$$D_i(t) = ||w_{\text{local},i}(t) - w_{\text{global}}(t)||_2 \tag{4}$$

The global penalty for unmitigated divergence across the fleet is defined as:

$$C_{\text{drift}}(t) = \sum_{i \in \mathcal{V}} D_i(t) \tag{5}$$

III.2.4. Learning Gain and Utility. To prevent trivial solutions where vehicles idle entirely to minimize costs, we introduce a performance-driven utility metric. The local learning gain is defined as the reduction in validation loss over a local validation set between consecutive epochs:

$$G_i(t) = L_i(t-1) - L_i(t) \tag{6}$$

To make forward-looking scheduling decisions, the loss function evolution can be approximated using a decaying exponential model:

$$L(e) \approx L_\infty + \alpha \cdot \exp(-\beta e) \tag{7}$$

The decision engine evaluates the immediate benefit of continued training via a utility function that weighs learning gain against model drift:

$$U_i(t) = \delta G_i(t) - \gamma D_i(t) \tag{8}$$

III.3. Optimization Objective

The objective is to maximize the overall learning gain of the fleet while penalizing communication overhead, runtime interference, and model divergence.

$$\max_{\{c_i(t), e_i(t), p_i(t)\}} \sum_{t=0}^{T-1} \Big[\delta \cdot \sum_{i \in \mathcal{V}} G_i(t) - \alpha C_{\text{net}}(t) - \beta C_{\text{comp}}(t) - \gamma C_{\text{drift}}(t) \Big] \tag{9}$$

where:

- δ scales the importance of local validation loss reduction.
- α controls sensitivity to communication overhead.
- β controls sensitivity to runtime interference.
- γ controls sensitivity to model divergence.

This optimization is subject to the following system constraints:

- **Compute Availability Constraint:** FL computation must not exceed available runtime slack:

$$c_i(t) \leq 1 - U_i^{\text{crit}}(t) \tag{10}$$

- **Training–Compute Coupling:** Local training capacity depends on available compute:

$$e_i(t) \leq \rho_i c_i(t) \tag{11}$$

  where ρ_i represents the maximum number of epochs supported per unit compute.
- **Network Capacity Constraint:** To prevent network congestion:

$$\sum_{i \in \mathcal{V}} p_i(t) M_i \leq B^{\text{cell}}(t) \tag{12}$$

- **Drift Stability Constraint:** To maintain acceptable learning performance:

$$D_i(t) \leq D_i^{\max} \tag{13}$$

IV. System Model

This section outlines the application of our mathematical model to the system's actual implementation logic and algorithmic design.

IV.1. Decision Variables and Operational States

At each time interval t, vehicle i determines its control variables, which effectively place the client in one of three operational states: CONTINUE, SYNC, or WAIT.

- **Compute Allocation ($c_i(t) \in [0, 1]$):** Represents the fraction of available compute resources allocated to FL processing.
- **Local Training Intensity ($e_i(t) \in \mathbb{Z}_{\geq 0}$):** Denotes the number of local training epochs performed.

- **Communication Decision ($p_i(t) \in \{0, 1\}$):** A binary variable indicating whether vehicle i transmits its model update.

The combination of these variables defines the vehicle's state:

- **CONTINUE** ($e_i > 0, p_i = 0$): The vehicle continues local training without synchronizing.
- **SYNC** ($p_i = 1$): The vehicle halts training and transmits its model update to the server.
- **WAIT** ($e_i = 0, p_i = 0$): The vehicle idles its FL process to conserve resources when bandwidth is poor or learning gains are marginal.

V. Evaluation Results

To determine the optimal configuration for our runtime policy engine, we conducted a comprehensive multi-dimensional hyperparameter sweep. As shown in Fig. 1, this sweep evaluated the impact of our primary weighting parameters (α, β, γ, and δ) against system metrics including network bandwidth usage, compute interference, model drift, and overarching utility.

The visual sweep indicates a strict requirement for a high learning gain reward (δ); traces achieving top-tier utility (approaching 14.5k) firmly converge between δ values of 24 and 29. Conversely, the high-utility paths do not bottleneck tightly on the α, β, or γ penalty axes. This broad distribution demonstrates that our proposed objective function is highly robust. These parameters can thus be flexibly tuned based on the physical constraints of the SDV environment without severely degrading system performance, provided that δ is sufficiently large to drive local validation loss reduction without overshadowing necessary constraints.

To better understand the physical trade-offs required to achieve peak performance, we analyzed the relationship between competing system metrics using a Pareto frontier analysis. As illustrated in Fig. 2, there is a distinct positive correlation between system utility and compute interference. The trial runs forming the optimal Pareto frontier (red dashed line) demonstrate that achieving utility scores above 10,000 strictly requires incurring higher interference levels (between 65 and 85).

This visual trend serves as direct physical proof of our system model's mathematical logic. Utility is driven by the learning gain ($G_i(t)$) generated during the *CONTINUE* state. To maximize this reward, the vehicle must execute intense local training epochs ($e_i > 0$). Because local training capacity is intrinsically tied to compute allocation ($c_i(t)$), maximizing learning gain inherently drives up the compute interference cost ($C_{\text{comp}}(t)$) imposed on co-located, safety-critical workloads. The Pareto frontier maps the absolute minimum acceptable interference necessary to reach a target utility under the optimized policy.

Fig. 3 visualizes the fundamental Federated Learning tension outlined in Section I: balancing model consistency against communication overhead. The Pareto frontier defines the optimal boundary for minimizing both model drift and network cost. It demonstrates a strict inverse relationship at the edge of performance: achieving the lowest possible model divergence (near 3,500) forces the system to incur a higher minimum network cost (approximately 411 MB/s). Restricting bandwidth usage to its lowest limits inherently allows the models to diverge further. This behavior confirms that our state machine operates precisely as intended, transitioning vehicles to the *SYNC* state ($p_i = 1$) only when necessary to cure divergence, thus establishing a hard, predictable relationship between bandwidth consumption and model accuracy.

Crucially, our broader analysis of the hyperparameter sweep revealed no meaningful correlation between total system utility and network cost. Trial runs achieving maximum utility were distributed uniformly across the entire spectrum of network usage. This lack of correlation acts as a primary success indicator for the proposed runtime engine. By allowing SDVs to confidently accumulate learning gains locally while absorbing moderate drift, the policy engine successfully decouples overarching FL performance from network saturation. SDVs can progress toward optimal model convergence internally without necessitating constant, bandwidth-prohibitive communication over the V2X network.

References

[1] Y. Hong, B. Lv, S. Wang, R. Wang, H. Tan, and F. C. Lau, "Fedcars: An efficient scheduling framework for in-vehicle federated learning integrated with carla-cosimulation," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 5, pp. 7997–8011, 2025.

[2] M. Karatas *et al.*, "Vr-vfl: Joint rate and client selection for vehicular federated learning under imperfect csi," 2026.

[3] J. Yan, T. Chen, Y. Sun, Z. Nan, S. Zhou, and Z. Niu, "Dynamic scheduling for vehicle-to-vehicle communications enhanced federated learning," *IEEE Transactions on Wireless Communications*, pp. 9373–9390, 2024.

[4] X. Qiang, Z. Chang, Y. Hu, L. Liu, and T. Hämäläinen, "Adaptive and parallel split federated learning in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4591–4604, 2025.

[5] L. Ballotta, N. D. Fabbro, G. Perin, L. Schenato, M. Rossi, and G. Piro, "Vrem-fl: Mobility-aware computation-scheduling co-design for vehicular federated learning," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3311–3326, 2025.

[6] X. Zhang, Z. Chang, T. Hu, W. Chen, X. Zhang, and G. Min, "Vehicle selection and resource allocation for federated learning-assisted vehicular network," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3817–3829, 2024.

[7] M. F. Pervej, R. Jin, and H. Dai, "Resource constrained vehicular edge federated learning with highly mobile connected vehicles," *IEEE Journal on Selected Areas in Communications*, pp. 1825–1844, 2023.

[8] S. Liu, G. Yu, R. Yin, J. Yuan, and F. Qu, "Communication and computation efficient federated learning for internet of vehicles with a constrained latency," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 1, pp. 1038–1052, 2024.

[9] X. Zhang, W. Chen, H. Zhao, Z. Chang, and Z. Han, "Joint accuracy and latency optimization for quantized federated learning in vehicular networks," *IEEE Internet of Things Journal*, vol. 11, no. 17, pp. 28876–28890, 2024.

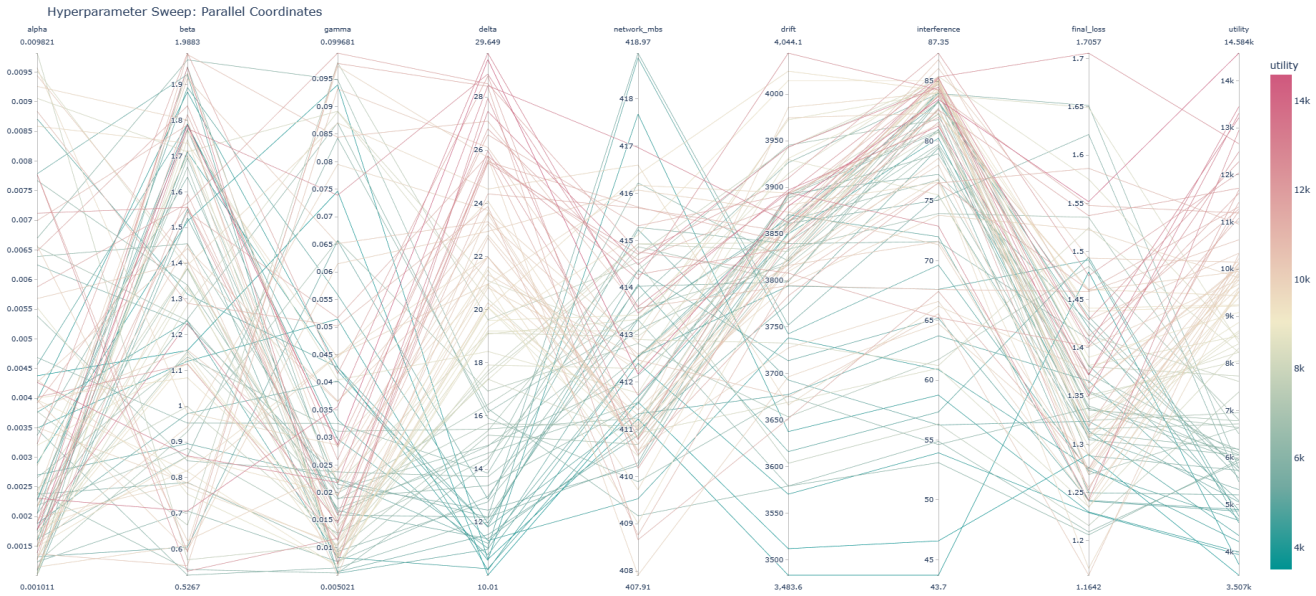


Figure 1. Parallel coordinates plot of the hyperparameter sweep. The highest performing trials (pink/red) indicate a strong convergence at the upper bounds of the δ and interference axes, while remaining robust across varying α, β, and γ penalty weights.

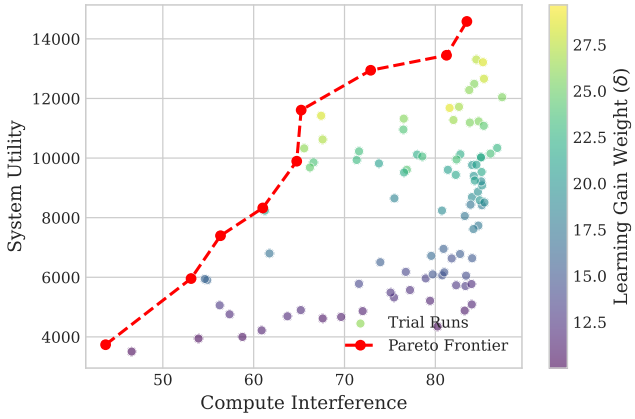


Figure 2. Trade-off between system utility and compute interference. The red dashed line denotes the Pareto frontier of optimal hyperparameter configurations, demonstrating that maximum utility inherently requires higher compute interference.

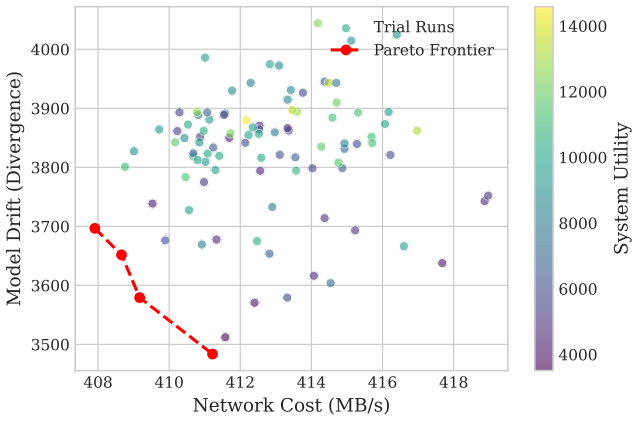


Figure 3. Trade-off between model drift (divergence) and network cost. The Pareto frontier illustrates the strict inverse relationship required to maintain model synchronization against available V2X bandwidth.

[10] M. Wu, M. Boban, and F. Dressler, "Flexible training and uploading strategy for asynchronous federated learning in dynamic environments," *IEEE Transactions on Mobile Computing*, pp. 12907–12921, 2024.

[11] H. Zhang, J. Bosch, and H. H. Olsson, "Real-time end-to-end federated learning: An automotive case study," in *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 459–468, 2021.

TABLE 1. SUMMARY OF VARIABLES AND CONTRIBUTIONS

Ref.	Compute Variables	Network Variables	Optimization Focus	Key Contribution
[1]	No explicit onboard hardware model	Uplink time and transmit power under mobility constraints	Minimize weighted upload delay and energy consumption	Formulates mobility-aware V2I model upload scheduling as an MDP and optimizes transmission time and power allocation
[2]	Limited explicit onboard compute modeling	Transmission rate, client selection, imperfect CSI, round duration	Balance FL convergence and round completion time under realistic wireless uncertainty	Proposes variable-rate vehicular FL with adaptive rate selection and client selection for faster convergence in mobile vehicular networks
[5]	Local compute scheduling, local training workload, latency budget	Predicted bitrate, transmission slot scheduling, mobility-aware radio maps	Jointly optimize learning progress, training time, and radio resource usage	Introduces predictive computation-scheduling co-design using radio environment maps for vehicular FL
[4]	CPU frequency, model partition point, energy budget, latency	Bandwidth allocation, transmit power, mobility-aware vehicle selection	Minimize training delay under energy and resource constraints	Tackles the VEC energy-delay tradeoff by dynamically shifting the model partition point based on available vehicle CPU and bandwidth.
[6]	Computation resource allocation, latency and energy constraints	Communication resource allocation, mobility-aware and channel-aware vehicle selection	Maximize model accuracy under latency and energy limits	Develops joint vehicle selection and compute-communication resource allocation for FL-assisted vehicular networks
[3]	Compute latency and energy are modeled, but onboard compute is not a main control variable	V2V relay selection, V2I and V2V transmission scheduling, transmit power, mobility-aware upload opportunities	Maximize successful model uploads and improve FL performance under mobility and energy constraints	Introduces V2V-enhanced dynamic scheduling for vehicular FL, using opportunistic relays to improve upload success and learning efficiency
[8]	Batch size, local computation burden, constrained latency	Spectrum allocation, channel-aware update transmission, compression ratio	Maximize convergence rate under a latency constraint by jointly tuning compute and communication parameters	Proposes joint optimization of compression ratio, batch size, and spectrum allocation to improve FL efficiency in IoV
[9]	No detailed hardware model, but learning-side cost is coupled to latency	Quantization level, wireless resource allocation, client selection, mobility-aware transmission	Balance model accuracy and latency through quantized update transmission and resource allocation	Develops a heterogeneous quantized FL framework that jointly optimizes update size and communication resources in vehicular networks
[10]	Local training effort adapts to link conditions, but no detailed CPU or GPU resource model	Upload timing, asynchronous communication, link-condition-aware transmission	Improve learning speed and reduce communication cost in dynamic wireless environments	Proposes a flexible asynchronous FL strategy that adapts training and uploading to communication quality and reduces unnecessary transmissions
[7]	CPU frequency, local iterations, energy budget, delay budget	RAT parameters, physical resource block allocation, transmit power, successful reception probability, sojourn time	Maximize successful model reception and learning progress under delay, energy, and cost constraints	Provides a strong joint formulation of local compute effort and wireless upload reliability for highly mobile vehicular FL
[11]	Heterogeneous embedded compute platform with GPU differences, but no formal compute optimization variables	Asynchronous update exchange, communication overhead, sliding-window data transfer	Improve training efficiency and robustness under hardware heterogeneity in automotive settings	Demonstrates real-time asynchronous federated learning in an automotive case study with reduced bandwidth cost and faster convergence than synchronous FL